

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

Answers

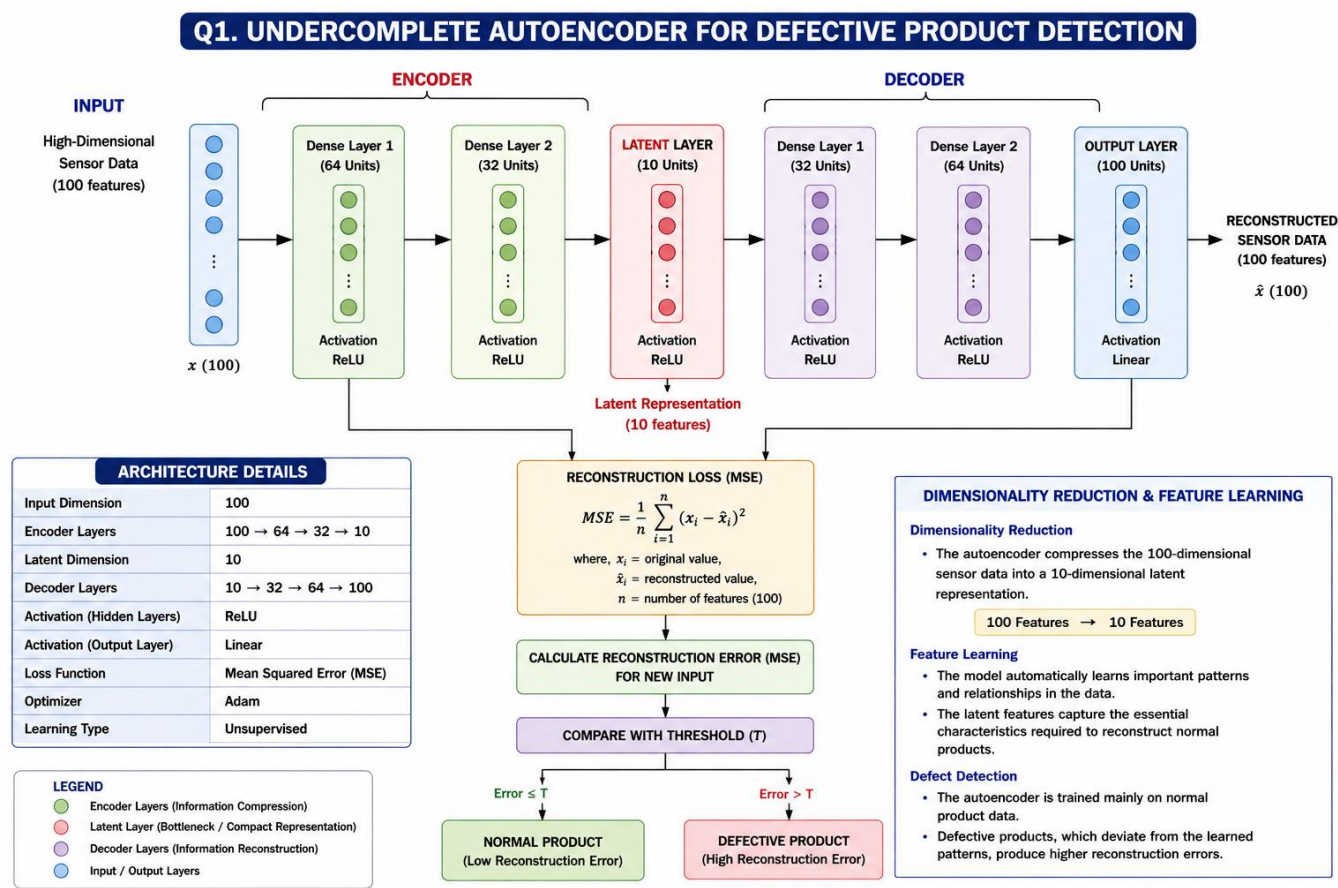
Q1. Design an Autoencoder-Based Feature Learning System

Problem Statement

A manufacturing company wants to detect defective products using high-dimensional sensor data. The data may contain temperature, pressure, vibration, speed, voltage, current, and other measurements.

An **Undercomplete Autoencoder** can learn a compact representation of the sensor data. It reconstructs the input, and the reconstruction error can be used to identify abnormal or defective products.

Architecture



Encoder

The encoder compresses the original data:

100 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 10

The 10-dimensional latent representation contains the important learned characteristics of the sensor data.

Decoder

The decoder reconstructs the original input:

[
10 $\rightarrow$ 32 $\rightarrow$ 64 $\rightarrow$ 100
]

The reconstructed data should be as close as possible to the original data.

where (x_i) is the original value and $(\hat{x}_i)$ is the reconstructed value.

Dimensionality Reduction and Feature Learning

The bottleneck reduces 100 features to only 10 features:

100 Features $\rightarrow$ 10 Latent Features

The model automatically learns important patterns without manually selecting features.

Defect Detection

New Sensor Data



Autoencoder



Reconstructed Data



Calculate MSE



Compare with Threshold

/ $\rightarrow$

Low Error High Error



Normal Defective

Evaluation

- Reconstruction MSE
- Accuracy

- Precision
- Recall
- F1-score

Conclusion

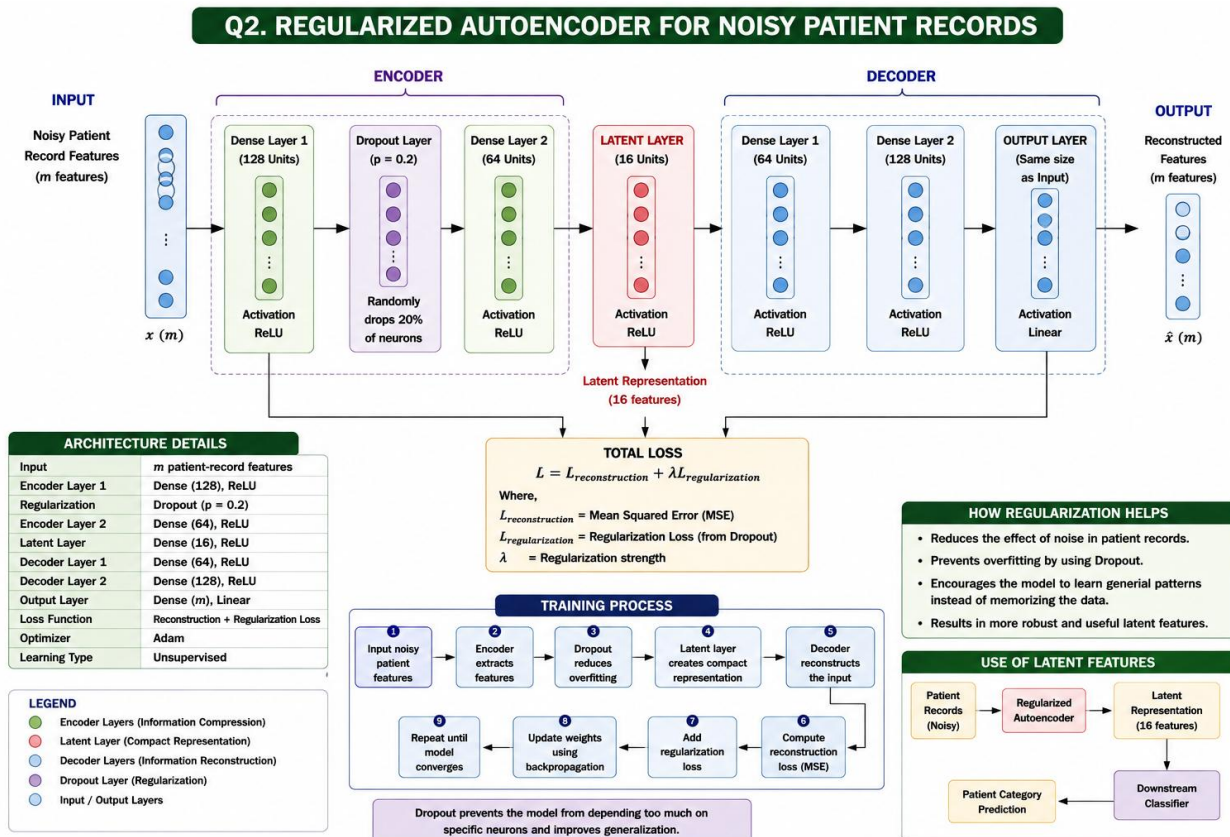
The Undercomplete Autoencoder provides **dimensionality reduction and automatic feature learning**. Normal products generally produce low reconstruction error, while defective products can produce higher error, allowing defects to be detected.

Q2. Design a Robust Regularized Autoencoder

Problem Statement

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. A **Regularized Autoencoder** can learn useful features while reducing the effect of noise and overfitting.

Proposed Architecture



Regularization Strategy

Dropout can be applied to the encoder. During training, some neurons are randomly deactivated.

For example:

Dense Layer



Dropout (0.2)



Next Layer

This prevents the network from depending too heavily on particular neurons.

Training Objective

The total objective can be written as:

$$L = L_{\text{reconstruction}} + \lambda L_{\text{regularization}}$$

The reconstruction loss ensures that useful information is retained, while regularization prevents excessive model complexity.

Learning Process

1. Input noisy patient features.
2. Encoder extracts important features.
3. Dropout prevents over-dependence on specific neurons.
4. Latent layer creates a compact representation.
5. Decoder reconstructs the input.
6. Reconstruction and regularization losses are calculated.
7. Weights are updated using backpropagation.
8. The process is repeated until the model learns a robust representation.

How It Reduces Overfitting

Regularization prevents the model from simply memorizing the training data. The learned representation focuses on important patterns rather than noise.

Use for Classification

Patient Records



Regularized Autoencoder



Latent Representation



Classifier



Patient Category Prediction

Evaluation

- Reconstruction error
- Classification accuracy
- Precision
- Recall
- F1-score
- Validation loss

Conclusion

A Regularized Autoencoder is suitable because it learns a **compact and robust representation** from noisy patient data while reducing overfitting. The latent features can then be provided to a downstream classification model.

Q3. Design a Deep Generative Model

Problem Statement

A recommendation or content-generation system needs to learn complex data distributions. A **Deep Belief Network (DBN)** can learn hierarchical representations using multiple hidden layers.

The assessment specifically allows either a **Deep Belief Network or Deep Boltzmann Machine** and asks for suitable visible and hidden layers, architecture, learning process, and justification.

An RBM contains:

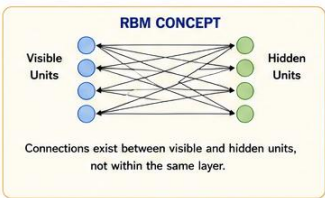
- There are connections between visible and hidden units, but no connections within the same layer.

● ● ● ● ●

$$\rightarrow \quad | \quad / \quad | \rightarrow \quad | \quad /$$

$\rightarrow \mid \mid \rightarrow \mid$

• • • • •



EXPECTED OUTCOME

The DBN learns hierarchical representations of user/item data and captures complex relationships. These representations are useful for accurate recommendations or content generation.



Hidden Layer

Why DBN is Suitable

A DBN is useful because it can learn **hierarchical features** from complex data.

For a recommendation system, it can learn patterns such as:

User Behaviour



Item Preferences



Hidden Relationships



User Preference Representation



Recommendation

Evaluation

- Recommendation accuracy
- Precision
- Recall
- F1-score
- RMSE for rating prediction

Expected Outcome

The DBN learns a deep representation of the data and can identify hidden relationships between users, items, or content.

Conclusion

A DBN is suitable for complex recommendation or content-generation problems because its stacked generative layers learn increasingly high-level representations from the input data.

Q4. Design a Stochastic and Contractive Representation Model

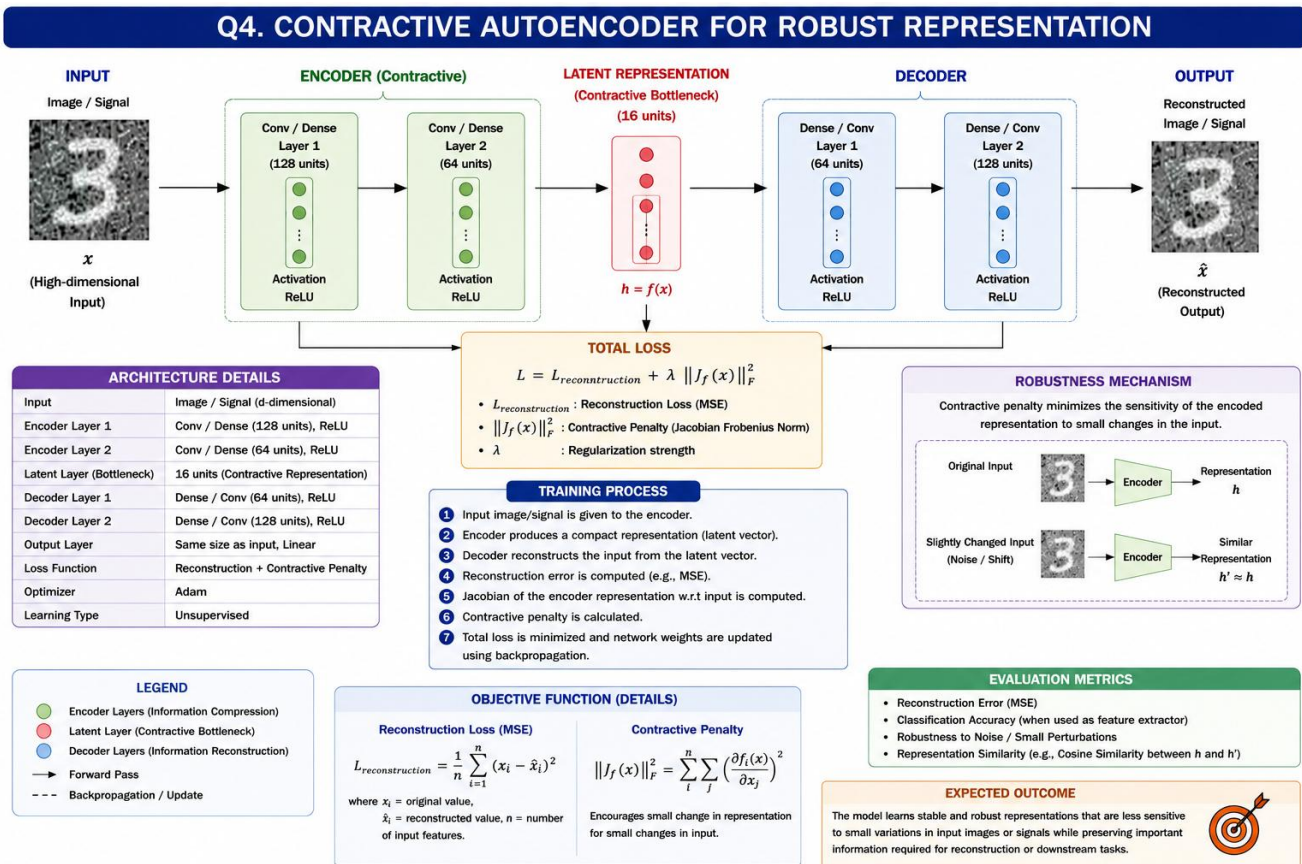
Problem Statement

A real-world image or signal-processing application requires representations that remain useful when small variations occur in the input. A **Contractive Autoencoder (CAE)** can be used to learn stable representations.

The assessment asks for a stochastic encoder/decoder or contractive encoder, architecture, objective function, robustness mechanism, and explanation of representation stability.

Proposed Contractive Autoencoder

Architecture



Contractive Objective

The objective can be represented as:

$$L = L_{\{reconstruction\}} + \lambda \|J_f(x)\|_F^2$$

where:

- $(L_{\{reconstruction\}})$ = reconstruction error

- $(J_{f(x)})$ = Jacobian of encoder representation
- $(\rightarrow \lambda)$ = regularization strength
- $(\|J_{f(x)}\|_F^2)$ = squared Frobenius norm

Robustness Mechanism

The contractive penalty discourages the encoder representation from changing too much when the input changes slightly.

For example:

Original Image



Encoder



Latent Representation

and

Slightly Changed Image



Encoder



Similar Latent Representation

Therefore, small input variations should produce similar learned representations.

Example

Suppose an image is slightly shifted or contains small signal noise:

Original $\rightarrow$ Encoder $\rightarrow$ Feature A

Modified $\rightarrow$ Encoder $\rightarrow$ Feature A'

The contractive objective encourages:

Feature $\rightarrow$ A $\rightarrow$ approx Feature $\rightarrow$ A'

Training Process

1. Provide image/signal as input.
2. Encoder extracts latent representation.
3. Decoder reconstructs the input.
4. Reconstruction error is calculated.
5. Contractive penalty is calculated.
6. Both losses are combined.
7. Network parameters are updated using backpropagation.
8. Training continues until stable representations are learned.

Evaluation

- Reconstruction error
- Classification accuracy using latent features
- Robustness to noise
- Representation similarity
- Accuracy under small input variations

Expected Outcome

The model learns representations that remain relatively stable when small variations occur in images or signals.

Conclusion

A Contractive Autoencoder is suitable for applications where **representation stability and robustness** are important. The contractive regularization reduces sensitivity to small input changes while preserving useful information.

Q5. Design a GAN-Based Data Generation System

Problem Statement

An organization has a limited number of training images and wants to generate realistic synthetic samples. A **Generative Adversarial Network (GAN)** uses a Generator and Discriminator to create realistic new images.

Discriminator

Input Image



Convolution

↓

Feature Extraction

Dense Layer

Sigmoid

↓

Real / Fake

Training Process

- 1. Generate random noise.
- 2. Generator produces a synthetic image.
- 3. Discriminator receives real images and fake images.
- 4. Discriminator learns to distinguish real from fake.
- 5. Generator learns to fool the Discriminator.
- 6. Both networks are updated.
- 7. Repeat the process.
- 8. Generated images gradually become more realistic.

Loss Functions

Discriminator loss:

$$L_D = -[\rightarrow \log D(x) + \rightarrow \log (1 - D(G(z)))]$$

Generator loss:

$$L_G = -\rightarrow \log D(G(z))$$

where (G(z)) is the generated image and (D) represents the Discriminator.

Evaluation

Measure	Purpose
Visual quality	Checks realism
FID	Compares real and generated distributions
Diversity	Checks variation among generated images
Discriminator performance	Measures real/fake discrimination
Downstream accuracy	Checks usefulness of synthetic data

GAN vs Autoencoder

Feature	GAN	Autoencoder
Main purpose	Generate new samples	Reconstruct input
Architecture	Generator + Discriminator	Encoder + Decoder
Input	Random noise	Original data
Output	New synthetic data	Reconstructed data
Training	Adversarial	Reconstruction-based
Loss	Adversarial loss	Reconstruction loss
Best suited for	Realistic data generation	Feature learning/compression

Why GAN is Suitable

GAN is suitable because the main requirement is to **generate realistic synthetic images** from a limited dataset. The Generator learns the data distribution while the Discriminator guides it toward producing increasingly realistic samples.

Expected Outcome

Limited Real Images



GAN



Generator learns distribution



Synthetic Images



Larger Training Dataset



Better Training of Target Model

Conclusion

A GAN provides an effective approach for generating realistic synthetic images when real training data is limited. The adversarial interaction between the Generator and Discriminator allows the system to progressively improve the quality of generated samples.